

GESTIÓN DEL PROCESO DE DESARROLLO DE SOFTWARE

Implementación de un pipeline CI/CD

API en FastAPI, Docker, GitHub Actions y despliegue en AWS EC2

Alumno: Jose Angel Gomez Castañon

Matrícula: 2023371222

Profesor: Emmanuel Martinez Hernandez

Fecha: 6 de octubre de 2026

01 Introducción

En esta práctica armé un flujo de integración y despliegue continuo para una API hecha con FastAPI. La idea es que cada vez que subo un cambio a la rama main, el código se pruebe solo, se empaquete en una imagen de Docker y se publique en un servidor de AWS sin que yo tenga que entrar a la máquina a hacer nada a mano.

Antes de llegar al despliegue tuve que cumplir con un mínimo de 70% de cobertura de pruebas, así que gran parte del trabajo fue ajustar los tests y la configuración de coverage. Al final también tuve varios errores con SSH y con la red de AWS, que explico en la sección de problemas.

Herramientas utilizadas

Herramienta	Para qué se usó
FastAPI	API REST con el endpoint /api/health
Pytest + coverage	Pruebas y medición de cobertura
Docker / Docker Hub	Imagen de la aplicación y registro
GitHub Actions	Automatización del pipeline
AWS EC2 (Ubuntu)	Servidor donde corre el contenedor

02 Pruebas y cobertura

Las pruebas se escribieron con Pytest. El pipeline falla si la cobertura baja de 70%, por lo que lo primero fue revisar qué archivos estaban bajando el porcentaje. Los módulos del servidor TCP y la generación de PDFs no forman parte de la API, así que los excluí en el archivo .coveragerc y marqué con pragma: no cover las partes que no se pueden probar de forma sencilla.

En GitHub Actions tuve que ejecutar las pruebas con `python -m pytest` en lugar de solo `pytest`, porque de la otra forma no encontraba los módulos del proyecto.

```
python -m pytest --cov=. --cov-fail-under=70
```

```
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\vrvang\cloud-computing-lab\workspace\webapp
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1, Faker-40.21.0, cov-7.1.0
collected 21 items

tests/test_main.py ..... [100%]

===== warnings summary =====
..\..\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\fastapi\testclient.py:1
C:\Users\vrvang\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

main.py:80
C:\Users\vrvang\cloud-computing-lab\workspace\webapp\main.py:80: DeprecationWarning:
  on_event is deprecated, use lifespan event handlers instead.

  Read more about it in the
  [FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

  @app.on_event("startup")

..\..\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\fastapi\applications.py:4744
C:\Users\vrvang\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\fastapi\applications.py:4744: DeprecationWarning:
  on_event is deprecated, use lifespan event handlers instead.

  Read more about it in the
  [FastAPI docs for Lifespan Events](https://fastapi.tiangolo.com/advanced/events/).

  return self.router.on_event(event_type) # ty: ignore[deprecated]

tests/test_main.py::test_healthcheck
C:\Users\vrvang\cloud-computing-lab\workspace\webapp\main.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  "timestamp": datetime.utcnow().isoformat(),

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform win32, python 3.14.5-final-0

Name           Stats    Miss  Cover   Missing
-----
database.py      30      12    60%   47-49, 52-59, 62
main.py          95       0   100%
tests\confest.py 32       0   100%
tests\test_main.py 140      0   100%
-----
TOTAL            297      12    96%
Required test coverage of 70% reached. Total coverage: 95.96%
===== 21 passed, 4 warnings in 1.50s =====
```

Figura 1. Resultado de pytest con el porcentaje de cobertura.

03 Contenedor con Docker

El Dockerfile parte de una imagen ligera de Python y usa dos etapas: en la primera se instalan las dependencias y en la segunda solo se copia lo necesario para correr la app. Además, el contenedor no corre como root sino con un usuario normal, y agregué un .dockerignore para no meter archivos que no hacen falta (venv, caché, etc.).

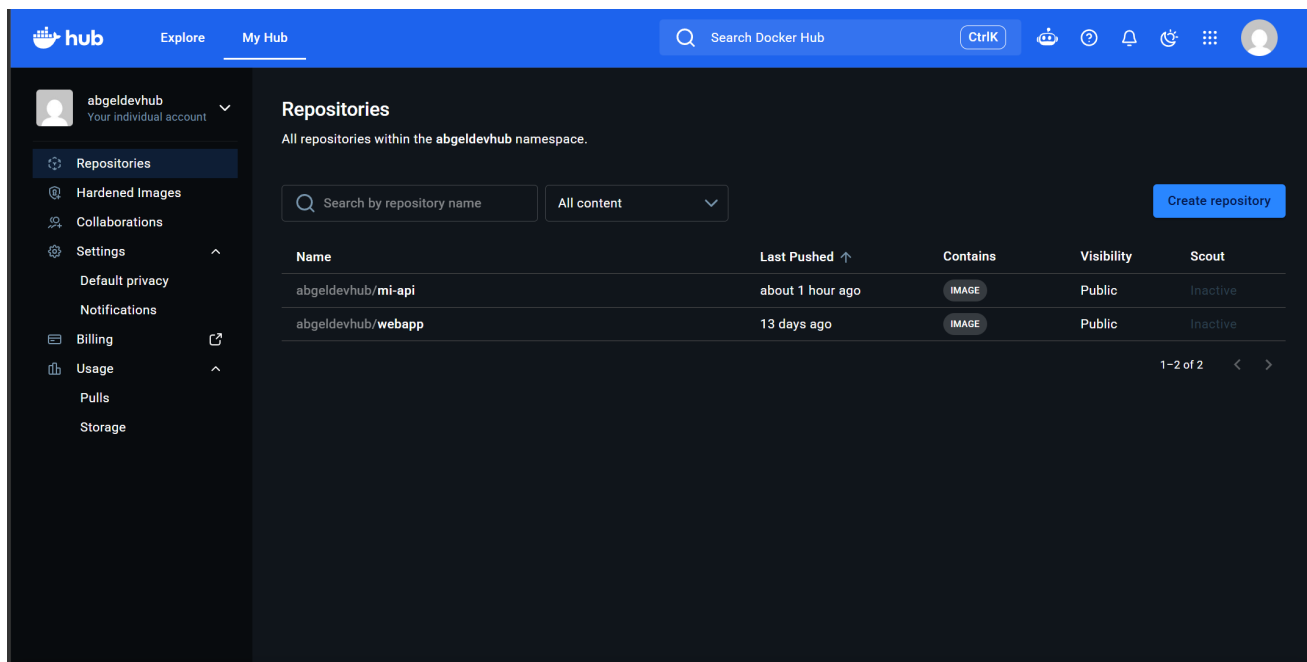


Figura 2. Imagen publicada en Docker Hub.

04 Pipeline en GitHub Actions

El workflow se dispara con cada push a main y tiene tres jobs que se ejecutan en orden. Si uno falla, los siguientes no corren.

Job	Qué hace
test	Instala dependencias y corre pytest con cobertura
build-and-push	Construye la imagen y la sube a Docker Hub
deploy-ec2	Se conecta por SSH a EC2 y actualiza el contenedor

Las credenciales (usuario y token de Docker Hub, IP del servidor, usuario y llave SSH) están guardadas como secretos del repositorio, no en el código.

Summary

All jobs

- test
- build-and-push
- deploy-ec2

Run details

- Usage
- Workflow file

Re-run triggered 1 hour ago

AngelDevHub -> d58f0b5 main

Status: Success

Total duration: 1m 6s

Artifacts: 1

deploy.yml

on: push

test 24s

build-and-push 22s

deploy-ec2 11s

build-and-push summary

Docker Build summary

For a detailed look at the build, download the following build record archive and import it into Docker Desktop's Builds view. Build records include details such as timing, dependencies, results, logs, traces, and other information about a build. [Learn more](#)

[AngelDevHub~fastapi-aws-pipeline~A4OMVX.dockerbuild](#) (42.49 KB - includes 1 build record)

Find this useful? [Let us know](#)

ID	Name	Status	Cached	Duration
A4OMVX	fastapi-aws-pipeline	completed	56%	6s

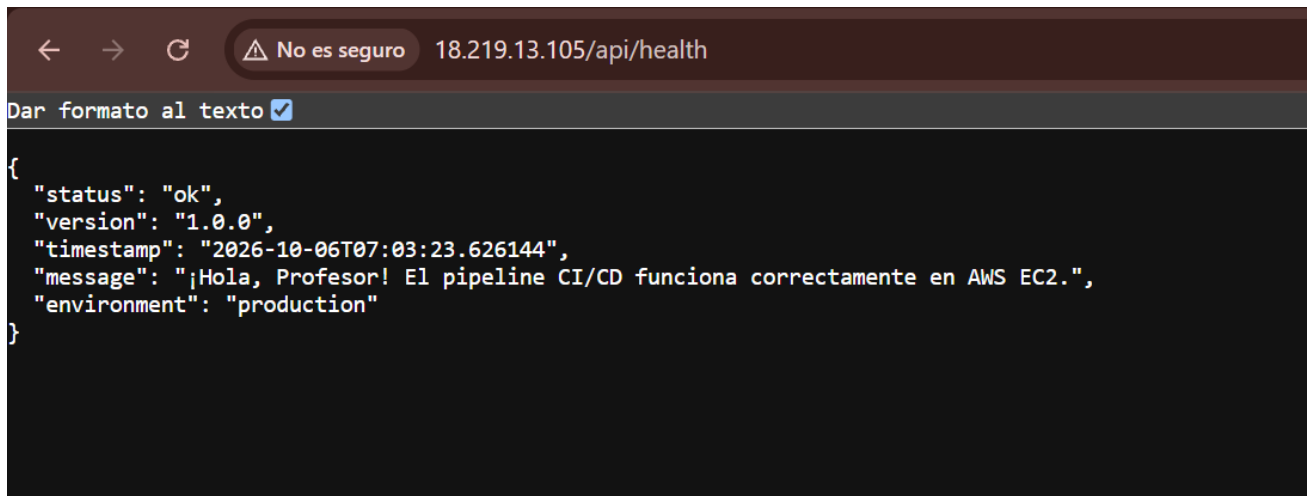
► Build inputs

Figura 3. Los tres jobs del pipeline completados en verde.

05 Despliegue en AWS EC2

El último job entra al servidor Ubuntu por SSH, descarga la imagen más reciente, detiene el contenedor anterior y levanta el nuevo en el puerto 80. Para poder ver la API desde el navegador hubo que abrir el puerto 80 en el Security Group de la instancia.

```
GET http://<IP-publica>/api/health
{"status": "ok", "environment": "production"}
```



```
{
  "status": "ok",
  "version": "1.0.0",
  "timestamp": "2026-10-06T07:03:23.626144",
  "message": "¡Hola, Profesor! El pipeline CI/CD funciona correctamente en AWS EC2.",
  "environment": "production"
}
```

Figura 4. Respuesta de la API en el navegador usando la IP pública de EC2.

06 Problemas que tuve

- Error de autenticación SSH: el usuario correcto en Ubuntu era 'ubuntu' y en el secreto había que pegar la llave .pem completa, con sus líneas BEGIN y END.
- Timeout al abrir la IP: faltaba permitir tráfico HTTP (puerto 80) en el Security Group.
- Pytest no encontraba los módulos en el pipeline: se arregló usando `python -m pytest`.
- La cobertura no llegaba al mínimo: se excluyeron los módulos ajenos a la API.

07 Conclusiones

Lo que más me sirvió de la práctica fue ver el flujo completo funcionando: un cambio en el código termina en producción en pocos minutos y con las pruebas ya validadas. También me quedó claro que la mayoría de los errores estuvieron en la configuración (secretos, red, permisos) y no en el código de la aplicación.